Advanced Database Systems Portfolio Summary

MIT 8103 | First Semester Continuous Assessment | CedarPay

Student: Chirim Amarachukwu **Student ID:** 301914084

GitHub repository: Advanced-Database-Portfolio--MIT8103 (grant the assessor access to the private repository before review).

Case study and database scope

CedarPay is a simulated Nigerian fintech platform for digital wallets and payments. Its database serves customers, operations and finance staff, and compliance reviewers. It stores customer and KYC records, accounts, beneficiaries, cards, transactions, ledger entries, fraud flags, audit events and login sessions. Key operations include onboarding and verifying customers, transferring funds, reconciling balances, reviewing fraud signals and auditing account activity.

Portfolio 1 - Relational database design

I designed the PostgreSQL relational foundation around eight core tables. The schema models customer identity and KYC, wallets, transactions, beneficiaries, cards, the double-entry ledger and fraud alerts. The scripts implement keys, foreign keys and integrity constraints, and the documentation explains normalization and controlled denormalization. The model and evidence are in the Portfolio 1 folder.

Portfolio 2 - Query processing and optimization

I evaluated eleven CedarPay queries using execution plans and measured results, then tested composite, partial and covering indexes and planner behavior. The analysis compares baseline and indexed plans, discusses selectivity and write/storage costs, and records the observed query trade-offs. SQL, plans, results and screenshots are in the Portfolio 2 folder.

Portfolio 1 ER model and Portfolio 2 query-plan evidence are also attached in this named submission folder.

Implementation and evidence

Portfolio 3 - Transactions and concurrency

I demonstrated COMMIT and ROLLBACK with dedicated CedarPay test accounts, then compared READ COMMITTED and REPEATABLE READ across two query sessions. The concurrency exercise reproduced a deadlock when sessions locked accounts in opposite orders. A consistent account-lock order allowed the competing transaction to wait and then proceed. The SQL scripts and screenshots are in the Portfolio 3 folder.

Portfolio 4 - NoSQL and advanced data models

I used MongoDB documents for flexible KYC profiles, audit events and login sessions while keeping balances and transfers in PostgreSQL. MongoDB validators rejected invalid KYC data with code 121. The seed script created 5,000 KYC profiles and 100,000 audit events. CRUD operations, four aggregation pipelines, indexes, unique-key rejection and TTL behavior were demonstrated. My additional aggregation ranked cities by successful logins. Evidence and outputs are in the Portfolio 4 folder.

Portfolio 5 - Distributed and cloud databases

I restored the CedarPay PostgreSQL dataset into a Docker primary and streaming read replica. The replica served read-only queries, showed lag while replay was paused, caught up after replay resumed, and was manually promoted during failover. Synchronous replication made a write wait while the standby was unavailable. The partitioning exercise copied 404,164 ledger rows into four hash partitions on account_id; EXPLAIN showed pruning for an account-key query and scans of all partitions when the key was omitted. This is a single-server partitioning demonstration, not multi-server sharding. In MongoDB Atlas, the three-member replica set acknowledged a majority write and returned the marker on a secondary read with majority read concern. My additional comparison found matching successful-login counts on primary and secondary reads. Evidence and scripts are in the Portfolio 5 folder.

Selected screenshots for all five portfolios are attached alongside this summary. The full set of scripts, outputs and screenshots is organized in the linked GitHub portfolio folders.

Evidence files attached

P1 ER model | P2 indexed query plan | P3 committed transfer | P4 index results | P5 Atlas replica metrics

Reflection and sources

Individual technical reflection

I enjoyed comparing MongoDB Compass with pgAdmin. Compass felt interactive for exploring documents, while pgAdmin felt more technical but let me run and manage SQL queries. The work clarified why CedarPay separates its data: PostgreSQL is suited to balances, transfers and ledger records that need relational constraints and transactions, while MongoDB accommodates KYC documents and audit events whose details vary. A customer ID shared across both systems is not automatically protected by a foreign key, so the application must validate and reconcile it. I also saw the availability and consistency trade-offs directly. An asynchronous replica can be behind the primary; synchronous replication can wait when a standby is unavailable. Hash partitioning helped when a query included account_id, but queries without that key touched every partition. I would measure more concurrent workloads and latency distributions before choosing production settings.

References

1. MIT 8103 Advanced Database Systems, Continuous Assessment, September Cohort 2026. Course brief supplied for this assessment.

2. PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Log-Shipping Standby Servers. Official documentation.

3. PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Table Partitioning. Official documentation.

4. MongoDB. Read Isolation, Consistency, and Recency. Official documentation.

5. MongoDB Atlas. Review Cluster Metrics. Official documentation.

6. Docker. Docker Compose documentation. Official documentation.

AI-use declaration

I used OpenAI ChatGPT for concept explanations, troubleshooting, code guidance and drafting support. I ran the database commands and scripts, checked the resulting outputs and screenshots, corrected instructions where they did not match my environment, and reviewed the final portfolio materials. I remain responsible for explaining and verifying the submitted work.

Submission note

This summary and the selected evidence use the required Surname_Firstname_StudentID naming pattern. Before submitting, confirm that the assessor can access the private GitHub repository.